

CS 236: Operating Systems Lab

Spring 2025-26

Lab 6: a process is nothing without memory

xv6 is a teaching operating system (developed at MIT) — an OS meant for teaching design and implementation concepts of an OS.

To install xv6 on Ubuntu, look up the details described in **Appendix A**.

The Holy Grail for xv6

- [xv6-risc book](#)
- [xv6-risc src booklet](#)

The xv6 source tree is organized to mirror OS subsystems:

- **kernel/** → process management, memory, file system, traps
- **user/** → user programs
- **mkfs/** → file system creation tool

Each directory corresponds to a logical OS component, enabling systematic exploration and modification. **You will mostly interact with the **kernel/** and **user/** directory.**

Submission Guidelines

Create a patch with:

```
Shell  
➤ ./patch.sh
```

Rename the generated **lab6.patch** to **<rollno>_lab6.patch** and submit it to Moodle

1. in the beginning, there was a process

This question requires modification of xv6's process creation facilities to implement a custom system call and investigate process hierarchy concepts.

This question explores **custom** process creation related possibilities.

1. a: one call, many processes!

Implement a new system call `tfork(int n, int *pids);` such that the calling process creates `n` child processes using a single call.

System call semantics

- `tfork()` returns:
 - `n` in the parent process
 - `0` in **each** child process
- The parent process collects the PIDs of all created child processes via a user memory region, provided to the system call via a pointer/user address argument.
- Child processes must not execute parent logic in the kernel.

Kernel-side instructions

- Implement `sys_tfork()` in `kernel/sysproc.c`
- Assign a system call number in `kernel/syscall.h`
- Register the system call in `kernel/syscall.c`
- Add a user-space wrapper in `user/usys.pl`
- Declare the function in `user/user.h`
- Use:
 - `argint()` and `argaddr()` to extract arguments
 - `kfork()` to create child processes
 - `copyout()` to copy child PIDs to user space
- Do **not** modify `kfork()`

User Program

- File name: `p1a.c` provided as template.
- The parent process must wait for all child processes
- Then the parent will print the process summary.
- All output must be printed **after** `wait()` to ensure deterministic output

Sample Output

```
None
$ p1a
child: pid=15 ppid=14
child: pid=16 ppid=14
child: pid=17 ppid=14
child: pid=18 ppid=14
parent: pid=14
Parent Process Summary:
  level 1 -> pid 15
  level 1 -> pid 16
  level 1 -> pid 17
  level 1 -> pid 18
```

Background

Each process in xv6 is represented by a Process Control Block (`struct proc`). This information is kernel-internal and must be exposed via a system call.

`myproc()` gives the kernel a pointer to the process that is currently running on the CPU. It is used to access the current process's PID, parent, page table, and other PCB fields. Most process-related system calls start by calling `myproc()`.

`copyout()` writes data from kernel variables into user-space memory safely. It translates a user virtual address using the process's page table before writing. Directly writing to user pointers is unsafe, so `copyout()` must be used.

`argint()` fetches an integer argument passed from user space to a system call. It reads the argument from the syscall argument registers and stores it in a kernel variable. It is typically used at the start of a syscall to read user-provided integers.

`argaddr()` fetches a user-space pointer argument passed to a system call. It copies the raw user virtual address into a kernel variable without dereferencing it. The kernel must later use `copyin()` or `copyout()` to safely access that address.

Read the functions `myproc()` from `proc.c`, `copyout()` from `vm.c` and `argint()` and `argaddr()` from `syscall.c`.

1. b: knowing one's origins (getppid)

xv6 does not provide a system call to retrieve the parent process ID. Implement a new system call — `int getppid(void);`

which returns the PID of the calling process's parent process.

User Program

- File name: `p1b.c` provided as template.
- Print both:
 - the process ID using `getpid()`
 - the parent process ID using `getppid()`

Sample Output

```
None
$ p1b
child: pid=15 ppid=14
child: pid=16 ppid=14
child: pid=17 ppid=14
child: pid=18 ppid=14
parent: pid=14
Parent Process Summary:
  level 1 -> pid 15
  level 1 -> pid 16
  level 1 -> pid 17
  level 1 -> pid 18
```

1. c: Rewriting the Family Tree (pfork)

Modify process creation semantics of **tfork** so that each newly created child becomes the parent of the next child, forming a **linear process chain**.

For `tfork2(4)`, the resulting process hierarchy should be:

None

`P → C1 → C2 → C3 → C4`

Kernel-side Instructions

- Implement `findproc()` in `kernel/proc.c` which takes a single argument **pid** to return struct `proc` of the process. See for `allocproc()` hints.
 - Find `kfork()` in `proc.c` and understand its implementation.
 - Implement `pfork()` by modifying `kfork()` such that:
 - **pfork** takes as argument the pid of a potential parent process
 - the parent process is resolved using `findproc()`
 - the **parent** pointer of the new process is explicitly assigned
 - Define these new functions in `defs.h`
 - Make `sys_tfork2()`
 - the first child's parent is the original process
 - each subsequent child uses the previously created child as its parent process
-

User Program

- File name: `p1c.c` provided as template.
-

Sample Output

None

`$ p1c`

```
child: pid=7 ppid=6
child: pid=6 ppid=5
child: pid=5 ppid=4
child: pid=4 ppid=3
parent: pid=3
Spawned Chain Summary:
  level 1 -> pid 4
  level 2 -> pid 5
  level 3 -> pid 6
  level 4 -> pid 7
```

Important Notes

- Do not modify trap handling or trampoline code
 - Do not introduce locks for user output
 - All synchronization must be done using `wait()`
-

2. Can we share a reality?

This question explores the concept of **shared memory** inside the xv6 operating system.

Shared memory is a mechanism that allows multiple processes or processes and the kernel to communicate efficiently by accessing the **same physical memory page** instead of copying data between processes. A unique physical page is mapped to potentially different virtual addresses of the user processes or to the kernel, for shared read/write access.

To implement shared memory, the kernel must:

- Allocate one physical page for sharing
- Keep track of that page inside the kernel
- Map the same physical page into the virtual address space of multiple processes and the kernel
- Safely detach and destroy the page when no longer needed

Lets implement this step-by-step.

Create the following files to implement the shared memory logic and primitives—

`kernel/shm.c`

`kernel/shm.h`

System calls to be implemented —

shm_init()

- Simulates initialization of shared memory
- In kernel
 - Allocate a page to be shared
 - Set reference count to 0
- FAILURE: If `refcnt >= 0`, shared memory is already initiated, return -1
- SUCCESS: return 0

shm_attach()

- In user — returns virtual address with shared memory mapping
- In kernel —
 - Finds VA region for process
 - Changes process size
 - Maps process VA range to shared PA page
 - Increments reference count
 - Update shared memory flag in PCB
- FAILURE: If `refcnt = -1` shared memory is not initiated return -1
- SUCCESS: return user VA to shared memory page

shm_detach() • In user — return success or failure • In kernel — - Check shared memory flag - Finds VA region for process - Change process size - UnMap process VA range to shared PA - pageDetach shared memory from process - Decrements reference count • FAILURE: If <code>refcnt = -1</code> , shared memory is not initiated return -1 • SUCCESS: return 0	shm_destroy() • Check reference count • FAILURE: If <code>refcnt != 0</code> Memory in use, return -1 • SUCCESS: If <code>refcnt == 0</code> return 0 • Reset refcnt to its default value <code>refcnt = -1</code> Free shared page from kernel
--	---

This lifecycle for initialization and usage of shared memory in each process is —
`init` → `attach` → `use` → `detach` → `destroy`

2.a: Before we share, let's count

Before implementing full memory mapping and sharing, this lab first simulates the **shared memory lifecycle using only a reference counter**.

shm.c provides the template for implementing shared memory system calls. Copy the provided template **shm.c** to **kernel/shm.c** and use the template to implement the syscalls required in this part.

To track of the details of the shared memory using the given structure provided in **shm.h**
Copy **shm.h** to **kernel/shm.h**

You do not have to update the **kva** variable in this part (defined as part of the struct in **shm.h**), just keep track of reference count.

To track whether a process has **attached** shared memory, you can add a variable to its PCB.

Before compiling, make sure `shm.o` is included in the Makefile.

Implement the **five** system calls, based on the logic explained below —

```
int shm_init(void)
uint64 shm_attach(void)
int shm_detach(void)
int shm_destroy(void)
int shm_refcnt(void) – returns value of the reference counter.
```


Note: For 2a. actual memory need not be allocated for sharing, but all the allocation, attaching and detaching checks are needed using the structure declared in shm.h

Sample Output

```
None
$ p2a
[Parent] Calling shm_init()...
[Parent] shm_init SUCCESS
[Parent] Initial Refcount: 0 (Expected: 0)
[Child 0] Calling shm_attach()...
[Child 0] Attached | Refcount: 1
[Child 1] Calling shm_attach()...
[Child 1] Attached | Refcount: 2

[Parent] Attempting shm_destroy() while refcount > 0...
[Parent] CORRECT: shm_destroy FAILED (refcount not zero)
[Child 0] Calling shm_detach()...
[Child 0] Detached | Refcount: 1
[Child 1] Calling shm_detach()...
[Child 1] Detached | Refcount: 0

[Parent] All children finished.
[Parent] Current Refcount: 0 (Expected: 0)
[Parent] Attempting shm_destroy() again...
[Parent] SUCCESS: shm_destroy worked (refcount = 0)
```

2.b: Making it real (mapping the shared page)

In this part, implement the **complete lifecycle** of shared memory inside the kernel.

- `shm_init()` → Create physical shared page
- `shm_attach()` → Map shared page to a process
- `shm_detach()` → Unmap shared page from a process
- `shm_destroy()` → Free the shared page.

For uniformity across all processes, map shared memory at a fixed user virtual address. Add the following line to your memory layout definition: **`#define SHMEM (TRAPFRAME - PGSIZE)`**

Kernel functions of interest — `kalloc`, `mappages`, `memset`, `uvmunmap`, `kfree`

Test your implementation using the given test codes **`p2b1.c`** and **`p2b2.c`**

Sample Output

```
Shell
$ p2b1
[Test 1] Attach without shm_init()...
PASS: shm_attach() correctly failed

[Test 2] Destroy without shm_init()...
PASS: shm_destroy() correctly failed

[Test 3] Detach before attach...
PASS: shm_detach() correctly failed before attach

[Test 4] Multiple detaches in same process...
First attach succeeded
First detach succeeded
PASS: Second detach correctly failed

[Test 5] Attach twice in same process...
PASS: Re-attach succeeded (should allow attach after detach)
PASS: Second attach correctly failed

Detaching shared memory...
SUCCESS

[Test 6] Destroy while child attached...
[Child] Attached to shared memory
[Parent] PASS: shm_destroy() failed while child attached
[Child] Detached successfully
[Parent] PASS: shm_destroy() succeeded after child detached
```

Sample Output

```
Shell
$ p2b2
[Child] shm_init succeeded
[Child] shm_attach succeeded
[Child] wrote: Message from child
[Child] shm_detach succeeded
[Parent] shm_attach succeeded
[Parent] reads: Message from child
[Parent] shm_detach succeeded
[Parent] shm_destroy succeeded
```

3. the kernel whisperer

Design a memory-mapped channel to enable communication between a user space process and the kernel. Instead of invoking system calls repeatedly, the user process and the kernel will communicate through a shared memory page. The kernel will service requests during timer interrupts without requiring additional syscalls.

Implement a minimal shared memory channel between a user process and the kernel. The user process should:

- Request the kernel to allocate and map one shared page
- Write a command into that page
- Wait for the kernel to respond
- Print the kernel's response

The kernel must service user requests on this channel during timer interrupts.

System call to implement — `uint64 kmap(void);`

This system call must:

- Allocate and map one physical page using `kalloc()`
- Zero the page using `memset()`
- Map the page into the calling process's address space
- Store a kernel address to the page in **struct proc**
- Return the user virtual address

Shared Page Format (template/part3/types.h)

User → Kernel Fields

- **u_cmd**: Contains the command number (e.g., `CMD_GETPID`), written by user and read by kernel
- **u_ready**: Set to `1` by user after writing `u_cmd`, read by kernel
- **u_ack**: Set to `1` by kernel after processing `u_cmd`, read by user

Kernel → User Fields

- **k_res**: Contains the result of the command, written by kernel, read by user
- **k_ready**: Set to `1` by kernel after writing `k_res`, read by user
- **k_ack**: Set to `1` by user after read `k_res`, read by kernel

Note: Copy the contents of `template/part3/types.h` to `xv6-riscv/kernel/types.h`

Service logic must be implemented inside the **usertrap()** function as part of timer interrupt handling. **trap.c** has more details.

First need to identify which of trap handling cases corresponds to the timer. Check the return values of the **devintr()** function.

Following enum-based commands are to be implemented in the kernel:

- **CMD_ECHO**: Return the magic number 42
- **CMD_GETPID**: Return the pid of the calling user process
- **CMD_GETPPID**: Return the ppid of the calling user process
- **CMD_UPTIME**: Return #ticks since machine start
- **CMD_PROC_COUNT**: Return #processes !UNUSED in system

Sample Output

None

\$ p3

Echoed: 42

PID: 3

PPID: 2

Uptime: 142

No of Processes: 5

Appendix A

Follow these steps for installing xv6 on a Ubuntu-based Linux machine:

1. Prerequisites
 - a. First, update your system:
sudo apt update
 - b. Install the required packages:
sudo apt install -y git build-essential qemu-system-riscv64 gcc-riscv64-unknown-elf
 - **build-essential**
For essential packages like libc, gcc, make, etc.
 - **qemu-system-riscv64**
Emulates a RISC-V machine in software.
Your computer's real CPU speaks x86-64, but xv6 is written for RISC-V.
QEMU **pretends to be a RISC-V computer**, so the OS can run even though the hardware is different.
 - **gcc-riscv64-unknown-elf**
It runs on your x86-64 machine but produces **RISC-V binaries**, not x86-64 ones.
We need this because xv6 must be compiled for the **CPU architecture that QEMU is emulating**.
2. Verify toolchain installation
riscv64-unknown-elf-gcc --version
qemu-system-riscv64 --version
If both commands work, your environment is ready.
3. Clone the xv6 RISC-V Repository
git clone <https://github.com/mit-pdos/xv6-riscv.git>
cd xv6-riscv
4. Build xv6
make
Run the command to build the OS. If compilation succeeds, you are ready to run xv6.
5. Run xv6 on QEMU
Start the xv6 operating system:
make qemu

Appendix B

A list of riscv ISA items and xv6 items that intersect with the xv6 system call implementation — For details of these and more look up Chapter 4 of the xv6 book.

C/C++

ecall

sret

stvec

sepc

scause

sstatus (SIE and SPP)

sscratch

satp

csrw

a0

TRAMPOLINE

TRAPFRAME

uservec

usertrap

userret

prepare_return

syscall

copyin

copyout

copyinstr

argint

argstr